Guion final del video NutriCare

Presentación técnica de React Native, Expo y Figma

Guion oral por diapositiva, notas de explicación sencilla y correcciones recomendadas para la presentación de Canva.

Duración objetivo: 7 minutos de explicación con diapositivas y 2 minutos de demostración de pantallas. Total aproximado: 9 minutos, a ritmo claro.

Presentación de referencia: NutriCare: presentación técnica, 14 diapositivas. Canva: https://www.canva.com/design/DAHYYYSbCRc/w8c8yFOMSdWW1aVBxU58gg/edit

Correcciones prioritarias antes de grabar

- Diapositivas 1 y 2: corregir espacios y el posible error “vn”; en la diapositiva 2 debe decir “ve”.
- Diapositiva 5: sustituir el árbol de carpetas por la estructura real de mobile/src.
- Diapositiva 6: mostrar las pestañas reales por rol. El paciente tiene Planes y Progreso, no una pestaña Diario.
- Diapositivas 7 y 8: usar los nombres que existen en el código y aclarar que Figma no se sincroniza automáticamente con los tokens del proyecto.
- Diapositiva 10: reemplazar NutritionContext y PlanContext por los cinco contextos reales.
- Diapositivas 11 a 13: presentar las plataformas como objetivo y la versión como prototipo; evitar llamar “robusta” a una función sin pruebas que lo sustenten.
- Diapositiva 14: añadir citas breves y dejar las referencias APA 7 completas en las notas de Canva o en este documento.

Guion por diapositiva

Lee el texto en voz natural. La explicación sencilla es una guía para comprender la idea; no hace falta leerla literalmente. La corrección señala qué conviene cambiar en Canva.

Diapositiva 1

Tiempo: 00:00-00:25

Título en pantalla: NutriCare: del diseño a una aplicación multiplataforma

Texto para decir: Hola, mi nombre es Fátima Oviedo. En esta presentación explicaré NutriCare, un prototipo académico para el seguimiento nutricional. Me enfocaré en cómo llevé una propuesta visual de Figma a una aplicación desarrollada con React Native y Expo, cómo se organiza el código y qué funciones ya se pueden demostrar. Al final mostraré los flujos principales en pantalla.

Explicación en sencillo: Abre con el propósito del video: explicar el paso del diseño a una aplicación interactiva, no solo enseñar pantallas.

Corrección recomendada: Sí. El texto extraído junta “app” y “multiplataforma”. Revisa el espacio y usa “pacientes” en minúscula dentro de la frase.

Texto sugerido para Canva: NutriCare: del diseño a una aplicación multiplataforma
React Native y Figma
Una aplicación para pacientes y nutricionistas

Diapositiva 2

Tiempo: 00:25-00:55

Título en pantalla: Dos perfiles de usuario

Texto para decir: NutriCare responde a dos necesidades. El paciente consulta su plan mensual, registra el seguimiento de sus comidas con fotografías y revisa su progreso. La nutricionista organiza pacientes, agenda, mediciones y comentarios. La aplicación ajusta la navegación según el rol. En esta versión académica, los datos de demostración se guardan localmente; todavía no se sincronizan entre usuarios por medio de un servidor.

Explicación en sencillo: El rol determina qué secciones aparecen. La información local sirve para demostrar el flujo, no para administrar expedientes clínicos reales.

Corrección recomendada: Sí. Corrige “vn” por “ve”. Separa “perfil” y “fotografías” en la lista del paciente y aclara que la información se adapta al rol.

Texto sugerido para Canva: PACIENTE: consultar plan, registrar comidas con fotos, revisar progreso y perfil.
NUTRICIONISTA: revisar pacientes, agenda, mediciones, planes y comentarios.
Cada rol ve las funciones que necesita. Prototipo académico con persistencia local.

Diapositiva 3

Tiempo: 00:55-01:30

Título en pantalla: Tecnologías y responsabilidades

Texto para decir: Cada tecnología cumple una tarea. Figma ayudó a definir la apariencia y los flujos. React Native permite construir la interfaz con componentes que se comparten entre plataformas. Expo aporta el marco de desarrollo y módulos para funciones del dispositivo. Expo Router organiza las pantallas usando archivos y carpetas. TypeScript describe la forma de los datos y ayuda a detectar incompatibilidades mientras se escribe el código.

Explicación en sencillo: Figma no convierte automáticamente el diseño en código: el diseño se traduce a componentes y estilos. Expo, React Native y Expo Router se complementan.

Corrección recomendada: Sí, leve. Cambia “UI nativa compartida” por “interfaz para móvil y web”. Presenta Expo como framework y herramientas de desarrollo, no solo ejecución.

Texto sugerido para Canva: Figma: diseño y flujos
React Native: interfaz multiplataforma
Expo: framework y módulos
Expo Router: navegación por archivos
TypeScript: tipos de datos

Conexión con la referencia: Base conceptual: React Native (s. f.) y Expo (s. f.).

Diapositiva 4

Tiempo: 01:30-02:00

Título en pantalla: Responsabilidades de la aplicación

Texto para decir: Podemos entender la aplicación en seis responsabilidades. Las pantallas reciben acciones; los componentes se reutilizan; los contextos organizan el estado; los esquemas validan formularios; los tipos describen la información; y AsyncStorage conserva ciertos datos en el dispositivo o navegador. Por ejemplo, al crear un paciente, la pantalla envía la información al flujo de validación, el contexto la actualiza y el almacenamiento la conserva localmente.

Explicación en sencillo: La interfaz es lo que la persona ve; el estado es lo que la aplicación recuerda mientras funciona; el almacenamiento local conserva algunos datos después de recargar.

Corrección recomendada: Aclaración recomendada. “Seis capas” describe responsabilidades lógicas, no seis carpetas ni una arquitectura de servidor. Di explícitamente que AsyncStorage es almacenamiento local.

Texto sugerido para Canva: Pantallas y rutas · componentes · estado · validación · tipos · persistencia local
AsyncStorage conserva datos en el dispositivo o navegador; no es una base de datos remota.

Diapositiva 5

Tiempo: 02:00-02:30

Título en pantalla: Estructura real del proyecto

Texto para decir: Esta es la estructura que encontramos en el proyecto móvil. Dentro de app están las rutas y pantallas. components/ui reúne elementos visuales reutilizables. constants contiene los valores del tema, context mantiene el estado por dominio, data incluye información de demostración, schemas contiene reglas de validación y types define los tipos compartidos. Si voy a cambiar una pantalla, empiezo por su archivo en app y sigo las dependencias que utiliza.

Explicación en sencillo: Las carpetas ordenan el trabajo: las pantallas no son lo mismo que los botones, los datos de prueba o las reglas de validación.

Corrección recomendada: Sí, importante. El árbol actual muestra “services”, “hooks” y “theme”, que no corresponden a las carpetas que existen. Sustitúyelo por este árbol real.

Texto sugerido para Canva: mobile/src/
app/ pantallas y rutas
components/ui/ interfaz reutilizable
constants/ colores y espaciado
context/ estado por dominio
data/ datos de demostración
schemas/ validación con Zod
types/ tipos de TypeScript

Diapositiva 6

Tiempo: 02:30-03:00

Título en pantalla: Navegación según el rol

Texto para decir: Expo Router construye las rutas a partir de los archivos de app. Los nombres entre paréntesis, como auth y tabs, agrupan pantallas sin agregarse a la dirección. El diseño raíz carga proveedores de estado y la navegación principal revisa el rol. El paciente ve Inicio, Planes, Progreso y Perfil. La nutricionista ve Inicio, Pacientes, Medición, Planes y Perfil. Comparten algunas secciones y cada rol recibe las que necesita.

Explicación en sencillo: Un archivo dentro de app suele corresponder a una ruta. El layout decide cómo se agrupan y presentan esas rutas.

Corrección recomendada: Sí, importante. Sustituye “Diario” por “Planes” en la navegación del paciente. La pestaña “Progreso” es la exclusiva del paciente; “Pacientes” y “Medición” son de la nutricionista.

Texto sugerido para Canva: PACIENTE: Inicio · Planes · Progreso · Perfil
NUTRICIONISTA: Inicio · Pacientes · Medición · Planes · Perfil
(auth) y (tabs) organizan rutas y layouts.

Conexión con la referencia: Base conceptual: Expo (s. f.), introducción a Expo Router.

Diapositiva 7

Tiempo: 03:00-03:25

Título en pantalla: Del diseño a valores reutilizables

Texto para decir: Figma sirvió como referencia para definir una apariencia coherente. En el código móvil, los colores, el espaciado, los radios y las sombras se reúnen en constants/theme.ts. A esos valores con nombre se les suele llamar tokens. Si una pantalla usa el color primario, toma el mismo valor que las demás. La transferencia se hizo llevando las decisiones al código; no existe una sincronización automática entre Figma y este archivo.

Explicación en sencillo: Un token es un valor visual con nombre, por ejemplo el verde principal. Reutilizarlo evita que cada pantalla invente un verde diferente.

Corrección recomendada: Sí. El código exporta colors, spacing, radius y shadow; no hay theme.type. Cambia “Card” por el nombre real “SurfaceCard” y evita sugerir sincronización automática con Figma.

Texto sugerido para Canva: Figma informa la decisión visual.
constants/theme.ts define colors, spacing, radius y shadow.
Los componentes aplican esos valores de forma consistente.

Conexión con la referencia: Base conceptual: Figma (s. f.), guía de variables y sistemas de diseño.

Diapositiva 8

Tiempo: 03:25-03:50

Título en pantalla: Componentes reutilizables

Texto para decir: La interfaz también repite patrones. El proyecto tiene PrimaryButton para acciones, SurfaceCard para agrupar información, FormField para entradas, BrandHeader para la marca, Screen para la estructura de página y BirthDatePicker para elegir fechas. Reutilizar estas piezas conserva estilos y comportamientos entre pantallas. Si cambiamos el componente, revisamos todas las pantallas donde aparece para confirmar que el cambio les funciona.

Explicación en sencillo: Un componente es una pieza de interfaz que se puede colocar en más de una pantalla, como un botón con estilo común.

Corrección recomendada: Sí. La lista actual mezcla nombres genéricos con piezas que no están en el código, como MetricTile, EmptyState y RoleTabs. Usa los nombres reales o aclara que son categorías conceptuales.

Texto sugerido para Canva: PrimaryButton · SurfaceCard · FormField
BrandHeader · Screen · BirthDatePicker
Menos estilos repetidos y cambios visuales más consistentes.

Conexión con la referencia: Base conceptual: Figma (s. f.), lección sobre componentes y sistemas de diseño.

Diapositiva 9

Tiempo: 03:50-04:25

Título en pantalla: Qué ocurre al guardar

Texto para decir: Tomemos como ejemplo el registro de un paciente. React Hook Form administra lo que se escribe en el formulario. Zod revisa las reglas y detecta datos inválidos. Si pasan la validación, PatientContext comprueba el expediente, añade el registro y lo guarda en AsyncStorage. Después actualiza el estado y la interfaz muestra el resultado. TypeScript ayuda a describir los datos, pero no reemplaza la validación de Zod.

Explicación en sencillo: El botón inicia el proceso, pero cada pieza hace un trabajo distinto: manejar campos, validar, actualizar información y guardarla localmente.

Corrección recomendada: Aclaración recomendada. Etiqueta el flujo como ejemplo de “guardar paciente”, para no afirmar que todos los botones usan exactamente la misma cadena. Separa visualmente cada paso con espacios.

Texto sugerido para Canva: Guardar paciente: formulario, validación con Zod, PatientContext, AsyncStorage e interfaz actualizada.
TypeScript describe los datos; Zod valida los valores ingresados.

Conexión con la referencia: Base conceptual: Microsoft (s. f.), documentación de TypeScript.

Diapositiva 10

Tiempo: 04:25-04:55

Título en pantalla: Contextos reales del proyecto

Texto para decir: Un contexto de React comparte información y operaciones entre pantallas sin tener que pasar cada dato manualmente por todos los componentes intermedios. En este proyecto existen AuthContext, PatientContext, MeasurementContext, PhotoContext y AppointmentContext. No hay NutritionContext ni PlanContext. Los planes de demostración están definidos en los archivos de data, así que conviene mostrar esa diferencia con exactitud.

Explicación en sencillo: Cada contexto mantiene una parte del estado. En este código se separan sesión, pacientes, mediciones, fotos y citas.

Corrección recomendada: Sí, importante. Reemplaza NutritionContext y PlanContext por MeasurementContext, PhotoContext y AppointmentContext. El proyecto actual utiliza cinco contextos.

Texto sugerido para Canva: AuthContext: sesión y rol
PatientContext: pacientes
MeasurementContext: mediciones
PhotoContext: fotos y comentarios
AppointmentContext: agenda
Los planes visibles usan datos de demostración, no PlanContext.

Diapositiva 11

Tiempo: 04:55-05:20

Título en pantalla: Código compartido y adaptación por plataforma

Texto para decir: La base de React Native se puede ejecutar en más de una plataforma, aunque cada sistema tiene detalles propios. En NutriCare usamos áreas seguras para evitar que el contenido quede debajo de la interfaz del teléfono, un contenedor que ayuda con el teclado y el selector de imágenes de Expo. La demostración de esta entrega se centra en la versión web. Antes de afirmar compatibilidad completa, también habría que probar los flujos en dispositivos iOS y Android.

Explicación en sencillo: Compartir código reduce duplicación, pero no sustituye las pruebas en cada dispositivo y sistema operativo.

Corrección recomendada: Sí, precisión. Presenta iOS, Android y web como plataformas objetivo; indica que la demo verificada es web, salvo que se hayan probado las versiones móviles. Evita atribuir gestos o permisos específicos si no los mostrarás.

Texto sugerido para Canva: Código compartido: componentes, tipos y reglas.
Adaptaciones: área segura, teclado y selección de imágenes.
Plataformas objetivo: Android, iOS y web. Demostración: web.

Conexión con la referencia: Base conceptual: React Native (s. f.) y Expo (s. f.).

Diapositiva 12

Tiempo: 05:20-05:55

Título en pantalla: Funciones demostrables del prototipo

Texto para decir: En el perfil de paciente se puede iniciar sesión, consultar el plan, agregar fotografías de comidas, revisar las fotos guardadas, consultar el progreso y editar el perfil. En el perfil de nutricionista se pueden revisar pacientes, gestionar consultas, registrar mediciones y responder comentarios sobre fotografías. La demostración debe distinguir entre una función interactiva y una conexión a un servicio real: los datos de este avance son locales o de ejemplo.

Explicación en sencillo: Enseña un flujo completo por perfil y aclara qué acciones guardan datos localmente.

Corrección recomendada: Sí. Cambia “Funcionalidad robusta” por “Funciones demostrables del prototipo”. “Robusta” afirma una calidad que no se prueba solo con una lista de pantallas.

Texto sugerido para Canva: PACIENTE: iniciar sesión, consultar plan, subir fotos, revisar progreso y editar perfil.
NUTRICIONISTA: revisar pacientes, agenda, mediciones, comentarios y perfil.
Prototipo académico con datos locales o de demostración.

Diapositiva 13

Tiempo: 05:55-06:25

Título en pantalla: Alcance actual y trabajo pendiente

Texto para decir: Es importante definir el alcance. NutriCare funciona como prototipo académico: permite recorrer los flujos y guardar algunos cambios localmente. No tiene todavía un backend que sincronice información entre dispositivos. Para usarlo con pacientes reales habría que implementar autenticación y almacenamiento seguros, proteger las fotografías, definir consentimiento y privacidad, y probar el sistema con rigor. Por eso no debemos presentar los datos de ejemplo como expedientes clínicos.

Explicación en sencillo: Una interfaz funcional no equivale a un producto clínico listo para producción.

Corrección recomendada: Ajuste recomendado. Cambia “Mañana” por “Para producción” y especifica que faltan backend, sincronización, protección de fotos y pruebas. Mantén visible que el estado actual es un prototipo.

Texto sugerido para Canva: ACTUAL: prototipo académico, roles demostrables y persistencia local.
PARA PRODUCCIÓN: backend seguro, privacidad y consentimiento, almacenamiento privado, sincronización y

pruebas.

Diapositiva 14

Tiempo: 06:25-07:00

Título en pantalla: Demostración funcional y referencias

Texto para decir: Para cerrar, conectaré las ideas con una demostración breve. Entraré con cada rol, recorreré sus pestañas y guardaré un dato de prueba. Después volveré a cargar la aplicación para comprobar qué información local permanece. Las referencias de React Native, Expo, TypeScript y Figma quedan documentadas al final del guion. NutriCare muestra cómo un diseño se puede convertir en una aplicación explicable, siempre distinguiendo lo demostrado de lo que todavía falta construir.

Explicación en sencillo: La demo sirve como evidencia del flujo. No la uses para afirmar sincronización o seguridad clínica que esta versión aún no ofrece.

Corrección recomendada: Sí. La diapositiva menciona referencias sin datos suficientes para ubicarlas. Añade citas breves en pantalla y deja las referencias completas APA 7 en las notas de Canva o en este documento.

Texto sugerido para Canva: Demostración: entrar por rol · recorrer pestañas · guardar un dato · recargar y verificar
Referencias abreviadas: React Native (s. f.), Expo (s. f.), Microsoft (s. f.) y Figma (s. f.).
Referencias completas: notas de Canva y guion del video.

Demostración de pantallas

Reserva dos minutos al final para mostrar el prototipo en vivo. Entra primero como paciente y luego como nutricionista. Usa cuentas de demostración y datos ficticios, no expedientes reales.

- 00:00-00:20 | Paciente: inicia sesión y muestra las pestañas Inicio, Planes, Progreso y Perfil.
- 00:20-00:50 | Planes: abre el menú del día, agrega una fotografía de comida y muestra dónde queda guardada.
- 00:50-01:00 | Cambia de sesión y entra con el perfil de nutricionista.
- 01:00-01:40 | Nutricionista: abre Pacientes, muestra Agenda o Medición y revisa el menú de comentarios de fotografías.
- 01:40-02:00 | Cierra con la persistencia local: recarga la app y explica que los datos de este prototipo no se sincronizan con un backend.

Términos técnicos en palabras sencillas

Componente: pieza reutilizable de interfaz, como un botón o una tarjeta

Ruta: dirección que identifica una pantalla

Estado: información que la aplicación mantiene disponible mientras funciona

Contexto: forma de compartir ese estado entre varias pantallas

Validación: comprobación de que los datos ingresados cumplen las reglas

Persistencia local: guardar datos en el dispositivo o navegador, sin servidor remoto

Token visual: valor con nombre para reutilizar un color, espacio o radio

Referencias en formato APA 7

Fuentes oficiales consultadas para sustentar las explicaciones técnicas. Las citas cortas pueden aparecer en la diapositiva 14; las referencias completas pueden dejarse en las notas del presentador.

Expo. (s. f.). *Create a project*. Expo Documentation. Recuperado el 19 de octubre de 2026, de https://docs.expo.dev/get-started/create-a-project/

Expo. (s. f.). *Introduction to Expo Router*. Expo Documentation. Recuperado el 19 de octubre de 2026, de https://docs.expo.dev/router/introduction/

Figma. (s. f.). *Guide to variables in Figma*. Figma Learn - Help Center. Recuperado el 19 de octubre de 2026, de https://help.figma.com/hc/en-us/articles/15339657135383-Guide-to-variables-in-Figma

Figma. (s. f.). *Lesson 3: Build your design system*. Figma Learn - Help Center. Recuperado el 19 de octubre de 2026, de https://help.figma.com/hc/en-us/articles/14548865734679-Lesson-3-Build-your-design-system

Microsoft. (s. f.). *The basics*. TypeScript documentation. Recuperado el 19 de octubre de 2026, de https://www.typescriptlang.org/docs/handbook/2/basic-types

React Native. (s. f.). *Introduction*. Recuperado el 19 de octubre de 2026, de https://reactnative.dev/docs/getting-started